

EXERCISES 04



AARHUS
UNIVERSITY

DEPARTMENT OF ELECTRICAL AND COMPUTER
ENGINEERING

19 AUGUST 2023

MICHEL VEDEL HOWARD
ASSISTANT PROFESSOR



EXERCISE 1

Consider a class Die that represents a die from some game say yatzy.

```
class Die{  
    public:  
        Die();  
        void roll();  
        int get_value() const;  
    private:  
        int value_;  
};
```

To make this work producing random data, a randomizer is needed



IN ALL CODING FROM NOW ON

Use the debugger as much as possible



EXERCISE 1

The <random> library in C++ gives us a randomizer

```
int main(){
    std::random_device rd_;// seed generator
    std::mt19937 mt_(rd_());//random generator seeded
    std::uniform_int_distribution<int> dist_(1, 6);// distribution definition
    for (size_t i = 0; i < 10; i++){
        std::cout << dist_(mt_) << " ";//generating random number from distribution
    }
    return 0;
}
```

You can test this your selves



EXERCISE 1

We want to start some sort of dice game and for that we need a die class

```
class die {  
public:  
    die();  
    void roll();  
    [[nodiscard]] int get_value() const;  
  
private:  
    int value_;  
};
```

But to make a randomized value of the die we need the objects

```
std::random_device rd_;  
std::mt19937 mt_{sd: rd_()};  
std::uniform_int_distribution<int> dist_{ a: 1, b: 6};
```

But since we do not want to replicate the in each object, we want them to be static members



EXERCISE 1.1

Implement the random generation objects as static members.

You are free to use <C++17 or >=C++17 techniques, but remember to use curly brace initialization if using >=C++ technique



EXERCISE 1.2

Implement the methods

`roll()`

`get_value()`

and test a die instance in the main function



EXERCISE 1.3

Now we want some more static behavior.

We want to record all rolls of all dice. That is, we want to record all rolls of all dice in the same data set.

We do that in a `vector<int>` where the vector is supposed to be of length 6 and each entry counts the number of 1... 6 rolls

Now, we **do not** want to store such data on each object so this must be static data

It should be initialized to size 6 and having start values 0.

- a. Implement the static `vector<int>` and initialize it.
- b. Furthermore, make the `roll` method insert appropriate values into the vector.

EXERCISE 1.4

Now we should use the vector for something

- a. Construct a *public static* function *frequencies* that returns a *vector<double>* containing the frequencies of every die value. For calculating the frequencies, you need the total number of throws. Construct a *private static* function *number_of_throws*, that calculates this for constructing the frequencies function.
- b. Every time you make a new throw you might want to log the mean values to monitor whether your die performs correctly. Construct a *public static* method *log_dice_rolls*.
- c. Make a run of say 100000 die rolls and log for every 1000 calls of roll.



EXERCISE 2

/*

* This function acts like a calculator producing the result of (first double operator second double)

* ex: 2.1 + 3.8

* The input numbers must be positive the operator can only be + - * / division by zero not defined

* All errors should be signaled by error codes somehow.

*/

double calculate(double , char, double);

The fact that the input values are positive does not mean that the out put is (having operator -)

So how do we signal errors codes??

Exercise 2.1

This evidently implies that we must change the function. We must some how return two values, which we cannot. So, a solution is to introduce a fourth l-value reference parameter that could be the result or the error. Try to define this prototype before looking at next page!!



EXERCISE 2

It seems that these two prototypes are possible

/*

- * This function acts like a calculator producing the result of (first double operator second double) ex: 2.1 + 3.8
- * The input numbers must be positive the operator can only be + - * / division by zero not defined
- * All errors should be signaled by error codes in the return value.
- * The result is passed to the caller by reference in the last parameter double&

*/

```
int calculate_1(double, char, double, double&);
```

/*

- * This function acts like a calculator producing the result of (first double operator second double) ex: 2.1 + 3.8
- * The input numbers must be positive the operator can only be + - * / division by zero not defined
- * All errors should be signaled to the caller by reference using the last parameter int &.
- * The result is returned as a double

*/

```
double calculate_2(double, char, double, int&);
```

EXERCISE 2

Exercise 2.2

Try to implement one of the solutions with error codes

Exercise 2.3

Construct a main function that reads input from the console and tests the calculator. It should translate all the error codes to suitable messages.

It is not a part of the exercise to validate that input is double char double

EXERCISE 3

Change the calculator into using exceptions



AARHUS
UNIVERSITY